Linear Regression Gradient Descent - Documented Recitation

In this recitation we implement one-dimensional linear regression from its mean-squared-error gradient, inspect convergence under several learning rates, and extend the same vectorized optimizer to a quadratic model.

Solution conventions

- The sample is deterministic, so numerical claims are reproducible.
- Custom functions are typed and keyword-only.
- The manual gradient implementation is vectorized with NumPy.
- sklearn is used as an independent optimum, not as a substitute for the requested derivation.
- MSE is evaluated after the final update, correcting the common off-by-one loss reporting in simple teaching implementations.

Imports

In [1]:
```
from __future__ import annotations

from matplotlib import pyplot as plt
import numpy as np
from numpy.typing import NDArray
import pandas as pd
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

from gradient_descent_init import LOGGER, RANDOM_SEED
from gradient_descent_utils import (
    FloatArray,
    GradientDescentResult,
    build_linear_design_matrix,
    build_quadratic_design_matrix,
    calculate_linear_loss_surface,
    calculate_linear_mse,
    calculate_maximum_stable_learning_rate,
    fit_linear_regression_gradient_descent,
    fit_quadratic_regression_gradient_descent,
)

LOGGER.info(
    msg=f"Gradient-descent recitation initialized with seed {RANDOM_SEED}."
)
```

```
2026-07-27 16:34:39,124 | INFO | gradient_descent_recitation | Gradient-descent r
ecitation initialized with seed 20260727.
```

Generate Data

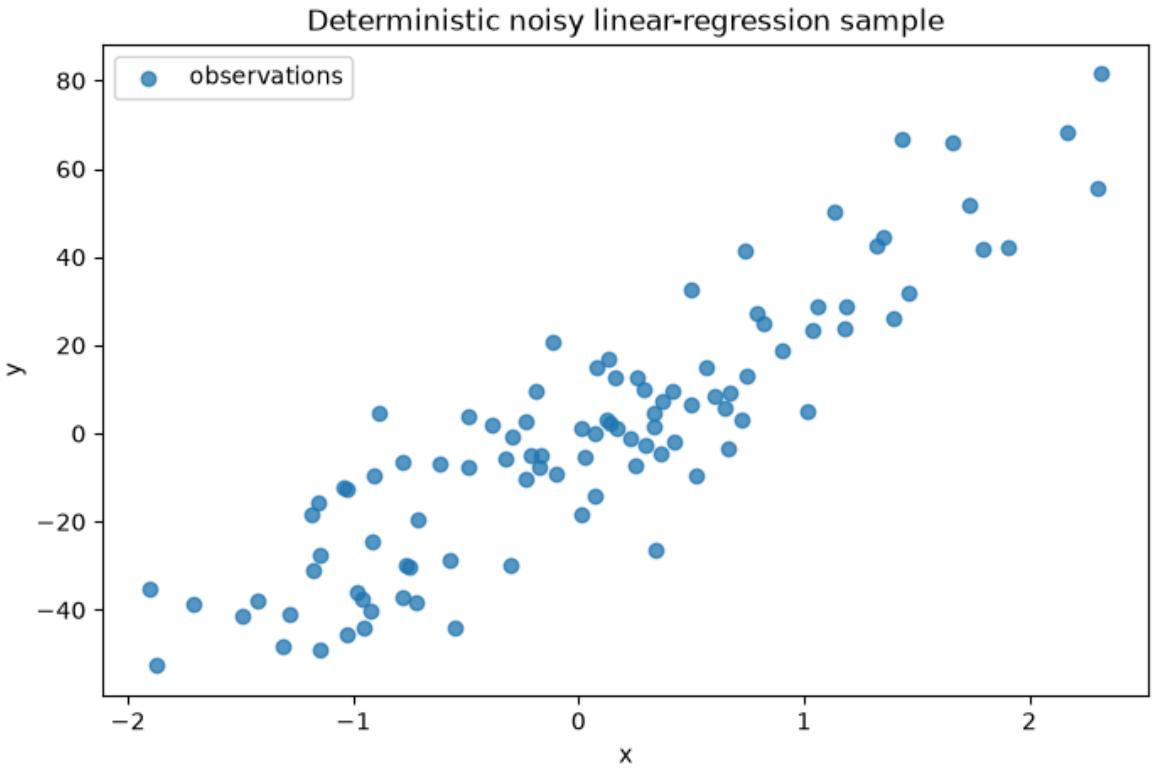
In [2]:
```
NOISE_STANDARD_DEVIATION = 15.0
NUMBER_OF_SAMPLES = 100

(
    feature_data,
    target_data,
    generating_slope_array,
) = make_regression(
    n_samples=NUMBER_OF_SAMPLES,
    n_features=1,
    noise=NOISE_STANDARD_DEVIATION,
    coef=True,
    random_state=RANDOM_SEED,
)
feature_data = np.asarray(
    a=feature_data,
    dtype=np.float64,
)
target_data = np.asarray(
    a=target_data,
    dtype=np.float64,
)
generating_slope = float(
    np.asarray(
        a=generating_slope_array,
        dtype=np.float64,
    ).reshape(-1)[0]
)

data_summary = pd.DataFrame(
    data=[
        {
            "number_of_samples": feature_data.shape[0],
            "number_of_features": feature_data.shape[1],
            "noise_standard_deviation": NOISE_STANDARD_DEVIATION,
            "generating_slope": generating_slope,
            "feature_mean": np.mean(a=feature_data[:, 0]),
            "target_mean": np.mean(a=target_data),
        }
    ]
)
data_summary.transpose()
```

Out[2]:

	0
number_of_samples	100.000000
number_of_features	1.000000
noise_standard_deviation	15.000000
generating_slope	27.312552
feature_mean	0.054550
target_mean	0.118421

Plot Data

In [3]:
```
data_figure, data_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
data_axes.scatter(
    x=feature_data[:, 0],
    y=target_data,
    alpha=0.75,
    label="observations",
)
data_axes.set(
    title="Deterministic noisy linear-regression sample",
    xlabel="x",
    ylabel="y",
)
data_axes.legend()
plt.show()
```



Implement Linear Regression

For observations (x_i, y_i), simple linear regression predicts $\hat{y}_i = mx_i + b$ and minimizes

$$L(m, b) = \frac{1}{N} \sum_{i=1}^{N} (mx_i + b - y_i)^2.$$

$$\frac{\partial L}{\partial m} = \frac{2}{N}\sum_i x_i(mx_i + b - y_i),$$
$$\frac{\partial L}{\partial b} = \frac{2}{N}\sum_i (mx_i + b - y_i).$$

One full-batch gradient step is $m \leftarrow m - \alpha\,\partial L/\partial m$ and $b \leftarrow b - \alpha\,\partial L/\partial b$.

Solution: implement simple linear regression

Three approaches considered

1. Use nested Python loops and scalar sums exactly as written in the formula.
2. Use a vectorized NumPy design matrix and full-batch gradient updates.
3. Call `scipy.optimize.minimize` or sklearn `LinearRegression` directly.

Choice: approach 2 is the most elegant match to this exercise. It exposes the gradient while replacing slow Python summation with optimized matrix operations. Approach 3 remains an independent correctness benchmark.

For design matrix $A = [x, 1]$ and coefficients $\theta = [m, b]^T$, the implementation uses $\nabla L = (2/N)A^T(A\theta - y)$ and $\theta \leftarrow \theta - \alpha\nabla L$. Histories contain the initial state and every updated state, and each stored loss is aligned to its parameters.

Direct answer: the blanks correspond to `prediction = m*x + b`, `cost = mean((prediction - y)**2)`, `m_gradient = (2/N) * sum(x * (prediction - y))`, `b_gradient = (2/N) * sum(prediction - y)`, followed by subtracting `learning_rate * gradient` from each parameter. Starting at $(m, b) = (10, -10)$ with $\alpha = 0.3$ for 50 updates gives $m = 27.122903$, $b = -1.361137$, and MSE 167.653725. sklearn's optimum is $m = 27.122903$, $b = -1.361137$, with MSE 167.653725; the agreement verifies the implementation. The data-generating slope was 27.312552, while noise explains why fitted OLS is not exactly equal to it.

In [4]:
```python
def linear_regression(
    *,
    feature_data: NDArray[np.float64],
    target_data: NDArray[np.float64],
    initial_slope: float = 0.0,
    initial_intercept: float = 0.0,
    number_of_epochs: int = 10_000,
    learning_rate: float = 0.0001,
) -> GradientDescentResult:
    """Fit y = m*x + b while retaining every optimization step.

    Args:
        feature_data: One numeric predictor column.
        target_data: Observed target values.
        initial_slope: Starting value for m.
        initial_intercept: Starting value for b.
        number_of_epochs: Number of full-batch gradient updates.
        learning_rate: Fixed positive step size.

    Returns:
```

```
        Coefficients [m, b] with aligned parameter and MSE histories.
    """
    return fit_linear_regression_gradient_descent(
        feature_data=feature_data,
        target_data=target_data,
        initial_slope=initial_slope,
        initial_intercept=initial_intercept,
        number_of_epochs=number_of_epochs,
        learning_rate=learning_rate,
    )
```

Do Regression

In [5]:

```
linear_result = linear_regression(
    feature_data=feature_data,
    target_data=target_data,
    initial_slope=10.0,
    initial_intercept=-10.0,
    learning_rate=0.3,
    number_of_epochs=50,
)
learned_slope = float(linear_result.coefficients[0])
learned_intercept = float(linear_result.coefficients[1])

sklearn_linear_model = LinearRegression()
sklearn_linear_model.fit(
    X=feature_data,
    y=target_data,
)
sklearn_prediction = sklearn_linear_model.predict(
    X=feature_data,
)
sklearn_slope = float(sklearn_linear_model.coef_[0])
sklearn_intercept = float(sklearn_linear_model.intercept_)
sklearn_mse = float(
    np.mean(
        a=np.square(sklearn_prediction - target_data),
    )
)

linear_model_comparison = pd.DataFrame(
    data=[
        {
            "method": "Vectorized gradient descent",
            "slope": learned_slope,
            "intercept": learned_intercept,
            "mse": linear_result.final_loss,
            "slope gap vs OLS": abs(
                learned_slope - sklearn_slope
            ),
        },
        {
            "method": "sklearn LinearRegression",
            "slope": sklearn_slope,
            "intercept": sklearn_intercept,
            "mse": sklearn_mse,
            "slope gap vs OLS": 0.0,
        },
```

```
    ]
).set_index(keys="method")
linear_model_comparison
```

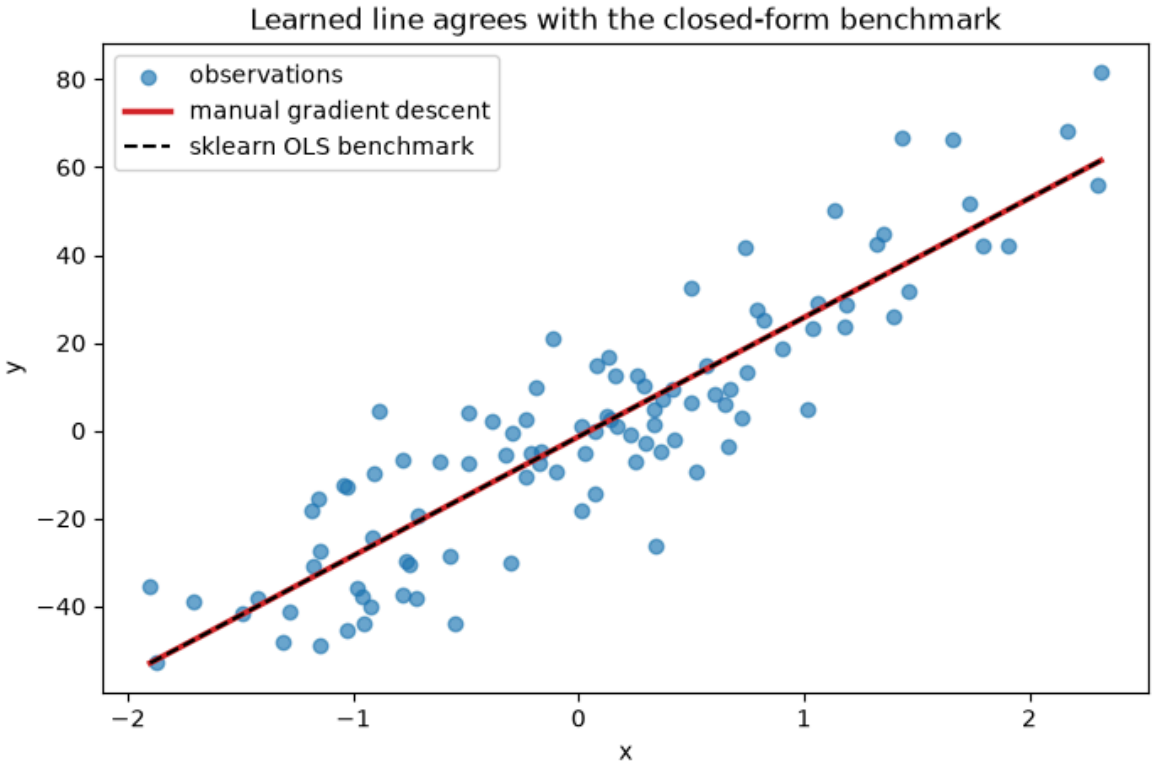
Out[5]:

method	slope	intercept	mse	slope gap vs OLS
Vectorized gradient descent	27.122903	-1.361137	167.653725	0.0
sklearn LinearRegression	27.122903	-1.361137	167.653725	0.0

Plot data and learned model line

In [6]:

```
line_predictor_values = np.linspace(
    start=float(feature_data[:, 0].min()),
    stop=float(feature_data[:, 0].max()),
    num=200,
)
learned_line = learned_slope * line_predictor_values + learned_intercept
sklearn_line = sklearn_slope * line_predictor_values + sklearn_intercept

line_figure, line_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
line_axes.scatter(
    x=feature_data[:, 0],
    y=target_data,
    alpha=0.65,
    label="observations",
)
line_axes.plot(
    line_predictor_values,
    learned_line,
    color="tab:red",
    linewidth=2.5,
    label="manual gradient descent",
)
line_axes.plot(
    line_predictor_values,
    sklearn_line,
    color="black",
    linestyle="--",
    linewidth=1.5,
    label="sklearn OLS benchmark",
)
line_axes.set(
    title="Learned line agrees with the closed-form benchmark",
    xlabel="x",
    ylabel="y",
)
line_axes.legend()
plt.show()
```



Visualizing Gradient Descent and learning rate

Now inspect how the parameter pair (m, b) moves over the loss surface and how the fixed learning rate controls convergence.

Create a reusable cost function for each value of m and b

Solution: reusable cost calculation

Three approaches considered

1. Square residuals in a Python list comprehension and call `sum`.
2. Use `numpy.mean(numpy.square(residuals))`.
3. Call sklearn `mean_squared_error`.

Choice: approach 2 is concise, allocation-aware, and naturally supports the vectorized loss-surface calculation. Approach 3 is used as a check.

Direct answer: for any candidate (m, b), compute $\hat{y} = mx + b$ and return $\text{mean}((\hat{y} - y)^2)$. At the learned parameters this returns 167.653725, the same value stored by gradient descent. The sklearn minimum is 167.653725.

In [7]:
```
def calc_cost(
    *,
    feature_data: NDArray[np.float64],
```

```
    target_data: NDArray[np.float64],
    slope: float,
    intercept: float,
) -> float:
    """Return the same mean squared error optimized by the model."""
    return calculate_linear_mse(
        feature_data=feature_data,
        target_data=target_data,
        slope=slope,
        intercept=intercept,
    )


cost_verification = pd.DataFrame(
    data=[
        {
            "calculation": "stored final GD loss",
            "mse": linear_result.final_loss,
        },
        {
            "calculation": "calc_cost at final GD parameters",
            "mse": calc_cost(
                feature_data=feature_data,
                target_data=target_data,
                slope=learned_slope,
                intercept=learned_intercept,
            ),
        },
        {
            "calculation": "sklearn OLS minimum",
            "mse": sklearn_mse,
        },
    ]
).set_index(keys="calculation")
cost_verification
```

Out[7]:

calculation	mse
stored final GD loss	167.653725
calc_cost at final GD parameters	167.653725
sklearn OLS minimum	167.653725

Do regression with different learning rates

In [8]:
```
learning_rates = np.asarray(
    a=[0.0001, 0.001, 0.01, 0.1, 0.5, 0.94],
    dtype=np.float64,
)
linear_design_matrix = build_linear_design_matrix(
    feature_data=feature_data,
)
maximum_stable_learning_rate = (
    calculate_maximum_stable_learning_rate(
        design_matrix=linear_design_matrix,
    )
```

```
)

learning_rate_histories: dict[float, GradientDescentResult] = {}
learning_rate_rows: list[dict[str, float | str]] = []
for learning_rate_value in learning_rates:
    current_learning_rate = float(learning_rate_value)
    current_result = linear_regression(
        feature_data=feature_data,
        target_data=target_data,
        initial_slope=10.0,
        initial_intercept=-10.0,
        learning_rate=current_learning_rate,
        number_of_epochs=50,
    )
    learning_rate_histories[current_learning_rate] = current_result
    bound_status = (
        "below bound"
        if current_learning_rate < maximum_stable_learning_rate
        else "at/above bound"
    )
    if bound_status == "at/above bound":
        LOGGER.warning(
            msg=(
                f"Learning rate {current_learning_rate:g} is not below "
                f"the stability bound {maximum_stable_learning_rate:.4f}."
            )
        )
    learning_rate_rows += [
        {
            "learning_rate": current_learning_rate,
            "bound_status": bound_status,
            "final_mse": current_result.final_loss,
            "mse_ratio_to_ols": current_result.final_loss / sklearn_mse,
            "slope": current_result.coefficients[0],
            "intercept": current_result.coefficients[1],
        }
    ]

learning_rate_summary = pd.DataFrame(
    data=learning_rate_rows,
).set_index(keys="learning_rate")
learning_rate_summary
```

Out[8]:

learning_rate	bound_status	final_mse	mse_ratio_to_ols	slope	intercept
0.0001	below bound	521.968231	3.113371	10.161991	-9.904782
0.0010	below bound	464.284120	2.769304	11.553630	-9.092236
0.0100	below bound	217.314371	1.296210	20.549883	-4.151064
0.1000	below bound	167.653726	1.000000	27.122309	-1.361004
0.5000	below bound	167.653725	1.000000	27.122903	-1.361137
0.9400	below bound	167.898958	1.001463	26.898260	-1.794760

Plot the loss and the learning curve

The figures below show both views requested by the exercise: trajectories of (m, b) on the loss surface and MSE versus epoch for each learning rate.

Solution: learning-rate behavior

Three approaches considered

1. Compare a fixed grid of rates only by plotting their paths.
2. Derive the stability bound from the largest Hessian eigenvalue.
3. Replace the fixed step with line search or an adaptive optimizer.

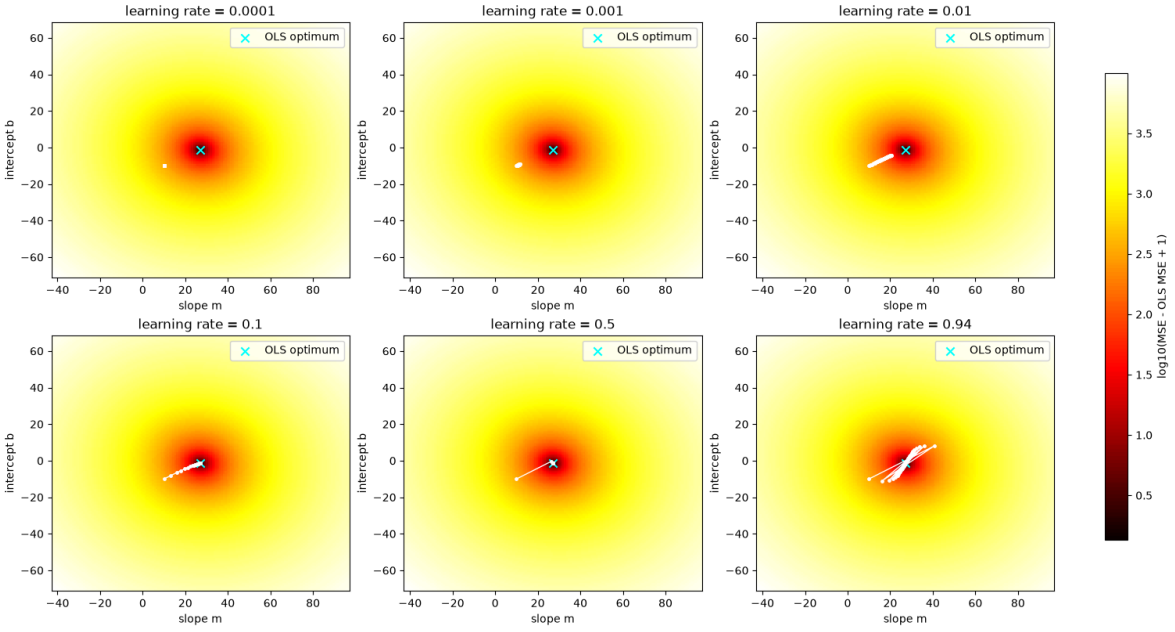
Choice: combine approaches 1 and 2. The requested plots show actual behavior, while the Hessian explains it. For this quadratic MSE, $H = (2/N)A^T A$ and fixed-step gradient descent requires $0 < \alpha < 2/\lambda_{\max}(H)$.

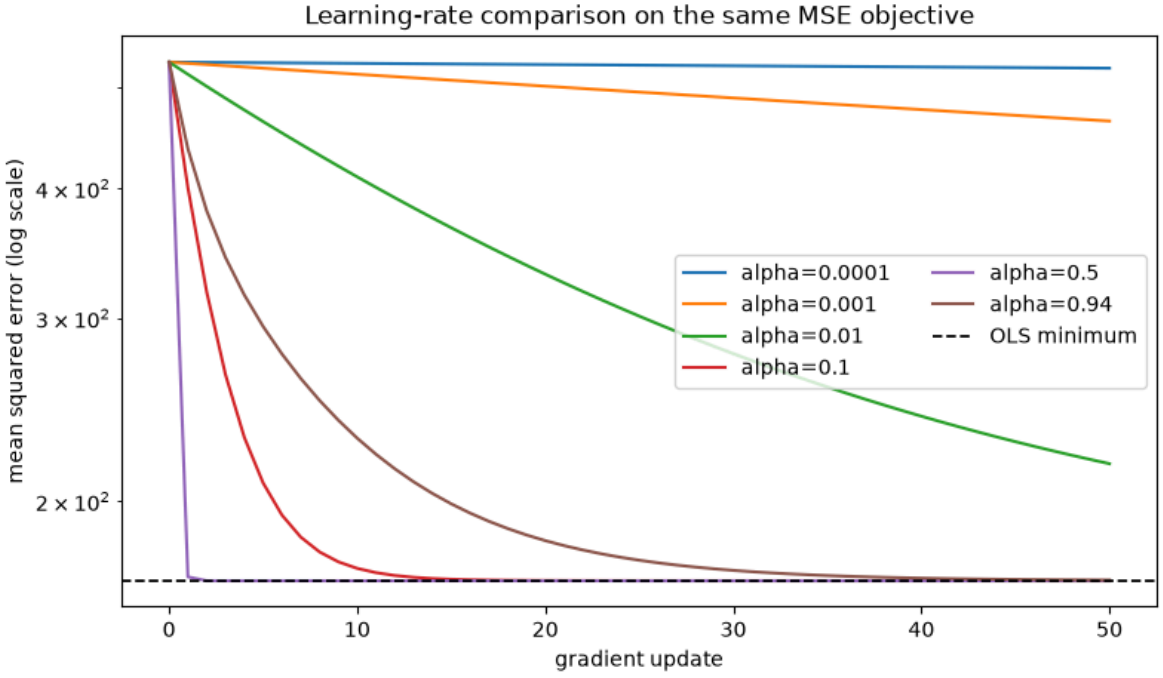
Direct answer: the deterministic design gives the strict upper bound $\alpha < 0.972516$. Every tested rate is below the theoretical fixed-step bound; the largest values can still oscillate while converging. Among the six requested values, `0.5` has the lowest MSE after exactly 50 updates (167.653725). Very small rates move safely but remain far from the optimum after 50 steps; larger stable rates arrive faster, while an excessive rate overshoots in alternating directions.

In [9]:
```
slope_plot_values = np.linspace(
    start=sklearn_slope - 70.0,
    stop=sklearn_slope + 70.0,
    num=160,
)
intercept_plot_values = np.linspace(
    start=sklearn_intercept - 70.0,
    stop=sklearn_intercept + 70.0,
    num=160,
)
loss_surface = calculate_linear_loss_surface(
    feature_data=feature_data,
    target_data=target_data,
    slope_values=slope_plot_values,
    intercept_values=intercept_plot_values,
)
shifted_loss_surface = np.clip(
    a=loss_surface - sklearn_mse,
    a_min=0.0,
    a_max=None,
)
log_loss_surface = np.log10(shifted_loss_surface + 1.0)

trajectory_figure, trajectory_axes = plt.subplots(
    nrows=2,
    ncols=3,
    figsize=(15, 8),
    constrained_layout=True,
)
```

```
loss_image = None
for plot_index, learning_rate_value in enumerate(learning_rates):
    current_learning_rate = float(learning_rate_value)
    current_axes = trajectory_axes.ravel()[plot_index]
    current_result = learning_rate_histories[current_learning_rate]
    loss_image = current_axes.imshow(
        X=log_loss_surface,
        cmap="hot",
        origin="lower",
        interpolation="nearest",
        extent=[
            float(slope_plot_values[0]),
            float(slope_plot_values[-1]),
            float(intercept_plot_values[0]),
            float(intercept_plot_values[-1]),
        ],
        aspect="auto",
    )
    current_axes.plot(
        current_result.coefficient_history[:, 0],
        current_result.coefficient_history[:, 1],
        color="white",
        marker="o",
        markersize=2.5,
        linewidth=1.0,
    )
    current_axes.scatter(
        x=[sklearn_slope],
        y=[sklearn_intercept],
        color="cyan",
        marker="x",
        s=55,
        label="OLS optimum",
    )
    current_axes.set(
        title=f"learning rate = {current_learning_rate:g}",
        xlabel="slope m",
        ylabel="intercept b",
    )
    current_axes.legend(loc="upper right")

if loss_image is None:
    raise RuntimeError("No learning-rate surface was rendered")
trajectory_figure.colorbar(
    mappable=loss_image,
    ax=trajectory_axes.ravel().tolist(),
    shrink=0.82,
    label="log10(MSE - OLS MSE + 1)",
)
plt.show()

learning_curve_figure, learning_curve_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(9, 5),
)
for learning_rate_value in learning_rates:
    current_learning_rate = float(learning_rate_value)
    current_result = learning_rate_histories[current_learning_rate]
    learning_curve_axes.semilogy(
```

```
        current_result.loss_history,
        label=f"alpha={current_learning_rate:g}",
    )
learning_curve_axes.axhline(
    y=sklearn_mse,
    color="black",
    linestyle="--",
    linewidth=1.25,
    label="OLS minimum",
)
learning_curve_axes.set(
    title="Learning-rate comparison on the same MSE objective",
    xlabel="gradient update",
    ylabel="mean squared error (log scale)",
)
learning_curve_axes.legend(
    ncols=2,
)
plt.show()
```

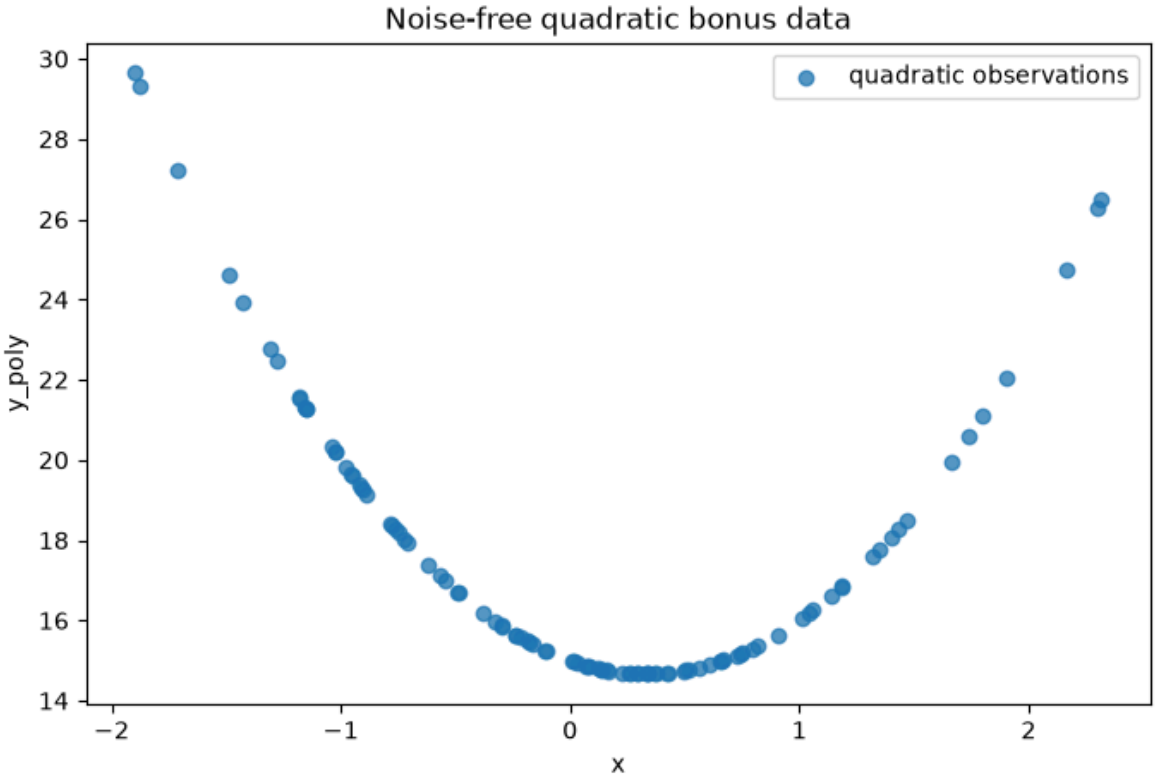




Bonus* - Polynomial regression

In [10]:
```
polynomial_target_data = (
    3.0 * np.square(feature_data[:, 0])
    - 2.0 * feature_data[:, 0]
    + 15.0
)
```

In [11]:
```
polynomial_data_figure, polynomial_data_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
polynomial_data_axes.scatter(
    x=feature_data[:, 0],
    y=polynomial_target_data,
    alpha=0.75,
    label="quadratic observations",
)
polynomial_data_axes.set(
    title="Noise-free quadratic bonus data",
    xlabel="x",
    ylabel="y_poly",
)
polynomial_data_axes.legend()
plt.show()
```



Modify the optimizer so that it returns a, b, and c minimizing

$$L(a, b, c) = \frac{1}{N} \sum_{i=1}^{N} \left(a x_i^2 + b x_i + c - y_i^{(\text{poly})} \right)^2.$$

Bonus solution: quadratic regression

Three approaches considered

1. Write three separate scalar gradient loops for a, b, and c.
2. Reuse the vectorized least-squares optimizer with columns $[x^2, x, 1]$.
3. Use sklearn `PolynomialFeatures` plus `LinearRegression`.

Choice: approach 2 is the cleanest extension because only the design matrix changes; the tested optimizer remains identical. Approach 3 verifies the recovered coefficients.

The gradients are $\partial L/\partial a = (2/N)\sum_i x_i^2(\hat{y}_i - y_i)$, $\partial L/\partial b = (2/N)\sum_i x_i(\hat{y}_i - y_i)$, and $\partial L/\partial c = (2/N)\sum_i(\hat{y}_i - y_i)$.

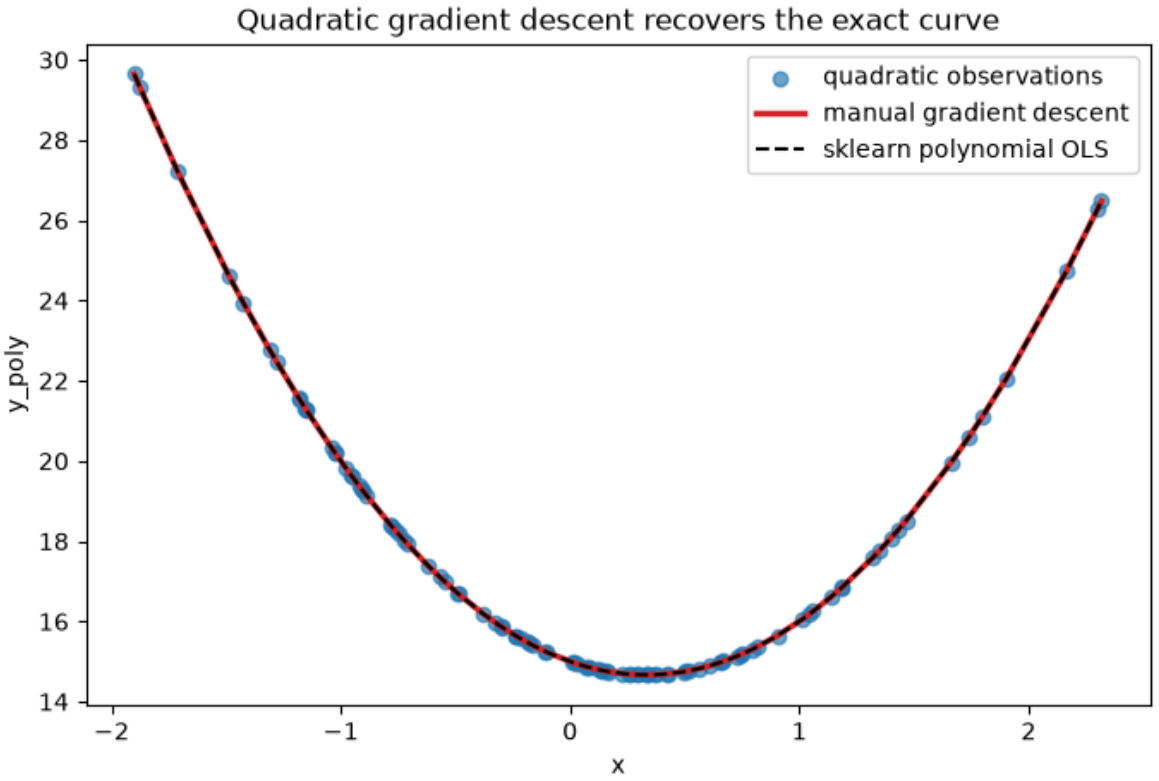
Direct answer: with the safe rate 0.181133 for 2,000 updates, the returned parameters are $a = 3.000000000$, $b = -2.000000000$, and $c = 15.000000000$, with final MSE $8.614e-30$. These recover the generating equation $3x^2 - 2x + 15$ to numerical precision.

In [12]:
```python
def polynomial_regression(
    *,
    feature_data: NDArray[np.float64],
    target_data: NDArray[np.float64],
    initial_quadratic_coefficient: float = 0.0,
    initial_linear_coefficient: float = 0.0,
    initial_intercept: float = 0.0,
    number_of_epochs: int = 10_000,
    learning_rate: float = 0.0001,
) -> GradientDescentResult:
    """Fit y = a*x**2 + b*x + c and retain the full history."""
    return fit_quadratic_regression_gradient_descent(
        feature_data=feature_data,
        target_data=target_data,
        initial_quadratic_coefficient=initial_quadratic_coefficient,
        initial_linear_coefficient=initial_linear_coefficient,
        initial_intercept=initial_intercept,
        number_of_epochs=number_of_epochs,
        learning_rate=learning_rate,
    )


quadratic_design_matrix = build_quadratic_design_matrix(
    feature_data=feature_data,
)
maximum_quadratic_learning_rate = (
    calculate_maximum_stable_learning_rate(
        design_matrix=quadratic_design_matrix,
    )
)
selected_quadratic_learning_rate = (
    0.5 * maximum_quadratic_learning_rate
)
polynomial_result = polynomial_regression(
    feature_data=feature_data,
    target_data=polynomial_target_data,
```

```
    initial_quadratic_coefficient=10.0,
    initial_linear_coefficient=-10.0,
    initial_intercept=10.0,
    learning_rate=selected_quadratic_learning_rate,
    number_of_epochs=2_000,
)
(
    learned_quadratic_coefficient,
    learned_linear_coefficient,
    learned_polynomial_intercept,
) = polynomial_result.coefficients

polynomial_transformer = PolynomialFeatures(
    degree=2,
    include_bias=False,
)
sklearn_polynomial_features = polynomial_transformer.fit_transform(
    X=feature_data,
)
sklearn_polynomial_model = LinearRegression()
sklearn_polynomial_model.fit(
    X=sklearn_polynomial_features,
    y=polynomial_target_data,
)

polynomial_comparison = pd.DataFrame(
    data=[
        {
            "parameter": "quadratic coefficient a",
            "generating_value": 3.0,
            "gradient_descent": learned_quadratic_coefficient,
            "sklearn": sklearn_polynomial_model.coef_[1],
        },
        {
            "parameter": "linear coefficient b",
            "generating_value": -2.0,
            "gradient_descent": learned_linear_coefficient,
            "sklearn": sklearn_polynomial_model.coef_[0],
        },
        {
            "parameter": "intercept c",
            "generating_value": 15.0,
            "gradient_descent": learned_polynomial_intercept,
            "sklearn": sklearn_polynomial_model.intercept_,
        },
        {
            "parameter": "final MSE",
            "generating_value": 0.0,
            "gradient_descent": polynomial_result.final_loss,
            "sklearn": np.mean(
                a=np.square(
                    sklearn_polynomial_model.predict(
                        X=sklearn_polynomial_features,
                    )
                    - polynomial_target_data
                ),
            ),
        },
    ]
).set_index(keys="parameter")
```

In [13]:

```
sorted_row_indices = np.argsort(
    a=feature_data[:, 0],
)
sorted_predictor_values = feature_data[sorted_row_indices, 0]
gradient_polynomial_prediction = (
    learned_quadratic_coefficient * np.square(sorted_predictor_values)
    + learned_linear_coefficient * sorted_predictor_values
    + learned_polynomial_intercept
)
sklearn_polynomial_prediction = sklearn_polynomial_model.predict(
    X=polynomial_transformer.transform(
        X=sorted_predictor_values[:, np.newaxis],
    ),
)

polynomial_fit_figure, polynomial_fit_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
polynomial_fit_axes.scatter(
    x=feature_data[:, 0],
    y=polynomial_target_data,
    alpha=0.65,
    label="quadratic observations",
)
polynomial_fit_axes.plot(
    sorted_predictor_values,
    gradient_polynomial_prediction,
    color="tab:red",
    linewidth=2.5,
    label="manual gradient descent",
)
polynomial_fit_axes.plot(
    sorted_predictor_values,
    sklearn_polynomial_prediction,
    color="black",
    linestyle="--",
    linewidth=1.5,
    label="sklearn polynomial OLS",
)
polynomial_fit_axes.set(
    title="Quadratic gradient descent recovers the exact curve",
    xlabel="x",
    ylabel="y_poly",
)
polynomial_fit_axes.legend()
plt.show()

LOGGER.info(
    msg="Gradient-descent recitation completed all exercises in order."
)
polynomial_comparison
```




```
2026-07-27 16:34:41,005 | INFO | gradient_descent_recitation | Gradient-descent r
ecitation completed all exercises in order.
```

Out[13]:

parameter	generating_value	gradient_descent	sklearn
quadratic coefficient a	3.0	3.000000e+00	3.000000e+00
linear coefficient b	-2.0	-2.000000e+00	-2.000000e+00
intercept c	15.0	1.500000e+01	1.500000e+01
final MSE	0.0	8.614361e-30	8.014827e-30